

C++&Qt day4 UDP 보고서 및 과제 1번 보고서

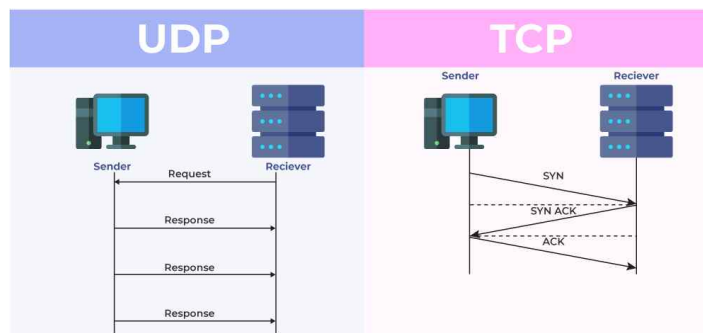
이름: 김도훈

학번: 2026406041

<목차>

1. UDP란 무엇인가?
 2. UDP 통신에서 IP 주소와 Port의 역할
 3. UDP의 데이터 전송 방식
 4. UDP와 TCP의 차이
 5. UDP의 장점과 단점
 6. UDP 통신을 공부하면서 배운점
-
2. 과제2
 - 2_1 과제2 설명
 - 2_3 과제2 구현
 - 2_3 실행 결과

1. UDP란 무엇인가



UDP는 User Datagram Protocol의 약자로 네트워크를 통해 데이터를 전송할 때 사용하는 통신 프로토콜 중 하나이다. 인터넷이나 같은 네트워크에 연결되어 있는 컴퓨터끼리 데이터를 주고받기 위해서는 데이터를 어떤 방식으로 전달할 것인지에 대한 규칙이 필요하다. 이러한 규칙을 프로토콜이라고 한다. UDP는 데이터를 빠르고 간단하게 전송하는 것을 목적으로 만들어진 프로토콜이다. TCP와 함께 네트워크 프로그램을 구현할 때 많이 사용되는 전송 계층 프로토콜이다. 컴퓨터끼리 네트워크를 통해 데이터를 주고받을 때 단순히 상대방의 IP 주소만 알고 있다고 해서 데이터를 바로 주고받을 수 있는 것은 아니다. 데이터를 어떤 방식으로 보내고 받을 것인지 정해야 하며 어떤 프로그램에서 해당 데이터를 처리할 것인지도 구분해야 한다. UDP에서는 IP 주소와 Port 번호를 사용하여 데이터를 보낼 컴퓨터와 프로그램을 구분한다. UDP의 가장 큰 특징은 상대방과 연결 상태를 먼저 만들지 않고 데이터를 바로 보낼 수 있다는 점이다. 예를 들어 컴퓨터 A에서 컴퓨터 B로 문자열 데이터를 전송한다고 생각할 수 있다. 컴퓨터 A에서 Hello라는 문자열을 전송하고 싶다면 상대방인 컴퓨터 B의 IP 주소와 Port 번호를 지정한 후 UDP 패킷을 전송한다.

TCP에서는 데이터를 보내기 전에 상대방과 연결하는 과정이 필요하다. 하지만 UDP에서는 이러한 연결 과정 없이 바로 데이터를 보낼 수 있다. 이 때문에 UDP를 비연결형 프로토콜이라고 한다. 비연결형이라는 것은 데이터를 보내기 전에 상대방과 연결을 설정하지 않는다는 의미이다. 데이터를 보내는 프로그램에서는 상대방의 IP 주소와 Port 번호만 알고 있으면 데이터를 전송할 수 있다. 따라서 UDP는 구조가 비교적 단순하며 빠른 데이터 전송이 필요한 프로그램에서 많이 사용된다. UDP 통신은 실시간으로 데이터를 빠르게 전달해야 하는 상황에서 많이 사용된다. 대표적으로 실시간 영상 전송이나 음성 통신 그리고 온라인 게임이나 센서 데이터 전송과 같은 분야에서 사용할 수 있다. 이러한 프로그램에서는 데이터 하나가 일부 손실되더라도 빠르게 다음 데이터를 전달하는 것이 더 중요한 경우가 많다. 예를 들어 실시간 영상에서 한 프레임이 제대로 전달되지 않았다고 하더라도 다음 영상이 바로 전달되기 때문에 전체 영상이 완전히 멈추는 것보다 계속 새로운 데이터를 전달하는 것이 더 중요할 수 있다.

UDP는 이러한 특징을 가지고 있기 때문에 실시간성이 필요한 프로그램에서 유용하게 사용할 수 있다.

2. UDP 통신에서 IP 주소와 Port의 역할

UDP 통신을 이해하기 위해서는 IP 주소와 Port 번호에 대해 이해해야 한다. IP 주소는 네트워크에 연결된 장치를 구분하기 위해 사용하는 주소이다. 네트워크에 여러 대의 컴퓨터가 연결되어 있을 경우 데이터를 어떤 컴퓨터에 보낼 것인지 구분할 방법이 필요하다. 이때 사용하는 것이 IP 주소이다. 예를 들어 다음과 같은 두 컴퓨터가 있다고 할 수 있다. 컴퓨터 A의 IP 주소 172.100.5.97 컴퓨터 B의 IP 주소 172.100.2.99 컴퓨터 A에서 컴퓨터 B로 데이터를 보내려면 컴퓨터 B의 IP 주소인 172.100.2.99를 목적지 주소로 사용하면 된다. 하지만 하나의 컴퓨터에서는 여러 프로그램이 동시에 네트워크를 사용할 수 있다. 예를 들어 한 컴퓨터에서 웹 브라우저가 실행되고 있을 수 있으며 UDP 채팅 프로그램도 실행되고 있을 수 있다. 또한 다른 네트워크 프로그램도 동시에 실행될 수 있다. 따라서 IP 주소만으로는 받은 데이터를 어떤 프로그램에게 전달해야 하는지 판단할 수 없다. 이때 사용되는 것이 Port 번호이다. Port는 하나의 컴퓨터 안에서 네트워크 데이터를 받을 프로그램을 구분하기 위해 사용하는 번호라고 생각할 수 있다. IP 주소는 데이터를 받을 컴퓨터를 찾는 역할을 하고 Port 번호는 해당 컴퓨터 안에서 데이터를 받을 프로그램을 찾는 역할을 한다. 쉽게 생각하면 IP 주소는 컴퓨터의 주소이고 Port 번호는 해당 컴퓨터 안에서 프로그램을 찾기 위한 번호라고 생각할 수 있다. UDP 프로그램을 구현할 때는 데이터를 보내는 기능뿐만 아니라 데이터를 받을 기능도 구현해야 한다. 데이터를 받는 프로그램에서는 자신이 사용할 Port 번호를 미리 정하고 해당 Port를 열어 놓아야 한다. 상대방이 해당 Port로 UDP 데이터를 전송하면 운영체제가 데이터를 받은 후 해당 Port를 사용하고 있는 프로그램에게 전달한다. 따라서 두 프로그램에서 서로 UDP 통신을 하기 위해서는 상대방이 어떤 IP 주소를 사용하고 있는지 알아야 하며 데이터를 받을 Port 번호도 알아야 한다.

3. UDP의 데이터 전송 방식

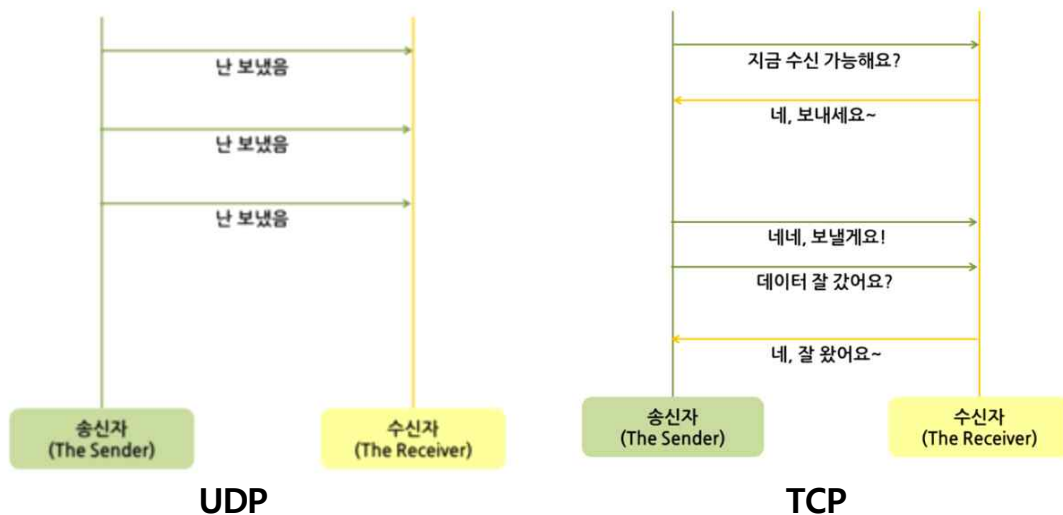
UDP에서 전송되는 데이터의 단위를 데이터그램이라고 한다. UDP는 데이터를 보낼 때 하나의 데이터그램을 만들어 상대방에게 전송한다. 데이터그램 안에는 실제로 보내고 싶은 데이터뿐만 아니라 통신에 필요한 정보도 같이 포함된다. 프로그램에서 Hello라는 문자열을 보낸다고 하면 프로그램은 해당 문자열을 네트워크에서 전송할 수 있는 형태의 데이터로 변환한다. 그 후 UDP를 이용하여 데이터를 전송한다. UDP 데이터가 전달되는 과정은 다음과 같이 생각할 수 있다.

1. 사용자가 문자열 입력
2. 프로그램에서 데이터 생성
3. UDP 데이터그램 생성
4. 상대방 IP와 Port 지정
5. 네트워크를 통해 전송
6. 상대 컴퓨터에서 데이터 수신
7. 상대 프로그램에서 데이터 확인

UDP에서는 데이터를 보낼 때 상대방이 실제로 실행 중인지 먼저 확인하지 않는다. 즉 데이터를 전송하는 프로그램은 정해진 IP 주소와 Port 번호를 이용하여 데이터를 바로 전송한다. 상대 프로그램이 실행되고 있지 않은 경우에도 송신 프로그램에서는 데이터를 전송할 수 있다. 하지만 상대방 프로그램이 해당 Port에서 데이터를 받고 있지 않다면 전달된 데이터를 프로그램에서 사용할 수 없다. 이러한 특징 때문에 UDP에서는 송신 측과 수신 측의 프로그램을 올바르게 설정하는 것이 중요하다. 위에서 설명했듯이 수신 측에서는 데이터를 받을 Port를 먼저 열어 놓아야 한다. 그 후 데이터가 들어올 때까지 기다린다. 송신 측에서 데이터를 전송하면 수신 측 프로그램은 들어온 데이터를 읽어서 사용한다. UDP는 하나의 데이터를 보낸 후 해당 데이터가 정상적으로 도착했는지 기본적으로 확인하지 않는다. 예를 들어 A 컴퓨터에서 B 컴퓨터에게 Hello라는 데이터를 전송했다고 하더라도 UDP 자체에서는 B 컴퓨터가 해당 데이터를 정상적으로 받았는지 확인하지 않는다. 따라서 네트워크 상태가 좋지 않은 상황에서는 데이터가 중간에 손실될 수 있다. 또한 여러 개의 데이터를 빠르게 전송했을 때 전송된 순서와 실제 도착 순서가 달라질 가능성도 존재한다. 예를 들어 다음과 같은 순서로 데이터를 전송했다고 할 수 있다.

첫 번째 데이터 A 두 번째 데이터 B 세 번째 데이터 C 전송한 순서는 A B C이지만 네트워크의 상황에 따라서 도착 순서가 달라질 가능성이 있다. UDP는 이러한 데이터를 다시 정렬하거나 누락된 데이터를 자동으로 다시 보내주는 기능을 기본적으로 제공하지 않는다. 이러한 부분은 UDP의 단점이라고 할 수 있지만 반대로 이러한 확인 과정이 없기 때문에 빠르고 단순하게 데이터를 전송할 수 있다는 장점이 생긴다.

4. UDP와 TCP의 차이



네트워크 통신을 구현할 때 UDP와 함께 많이 사용하는 프로토콜이 TCP이다.

UDP와 TCP는 모두 컴퓨터끼리 데이터를 주고받을 때 사용하지만 데이터를 처리하는 방식에는 차이가 있다. TCP의 가장 큰 특징은 통신을 시작하기 전에 상대방과 연결 상태를 만든다는 점이다. TCP 통신에서는 먼저 상대방과 연결이 가능한지 확인하고 연결이 정상적으로 이루어진 후 데이터를 주고받는다. 또한 전송한 데이터가 정상적으로 도착했는지 확인하고 문제가 발생한 경우 데이터를 다시 전송하는 기능도 제공한다. 이 때문에 TCP는 데이터를 정확하게 전달해야 하는 프로그램에서 많이 사용된다. 예를 들어 파일을 전송할 경우 데이터 일부가 사라지면 파일이 정상적으로 만들어지지 않을 수 있다. 이러한 경우에는 속도도 중요하지만 데이터가 정확하게 전달되는 것이 더 중요하다. 따라서 이러한 통신에서는 TCP가 적합할 수 있다. 반대로 UDP는 연결 과정을 먼저 만들지 않는다. 상대방의 IP 주소와 Port 번호를 알고 있다면 바로 데이터를 보낼 수 있다. 또한 전송한 데이터가 정상적으로 도착했는지 확인하는 과정도 기본적으로 없다. 따라서 구조가 단순하며 TCP에 비해 빠르게 데이터를 전송할 수 있다.

TCP의 동작을 간단하게 나타내면 다음과 같다.

상대방과 연결 요청 -> 연결 확인 -> 연결 완료 -> 데이터 전송-> 데이터 도착 확인

반면 UDP는 다음과 같이 동작한다.

데이터 생성 -> 상대방 IP와 Port 지정 -> 데이터 전송

이 차이 때문에 UDP는 실시간성이 중요한 프로그램에 적합하다. 예를 들어 실시간 게임에서 캐릭터의 현재 위치를 계속 전송한다고 생각할 수 있다. 캐릭터가 이동할 때마다 새로운 위치 정보가 계속 만들어진다. 이때 이전 위치 데이터 하나가 전달되지 않았다고 해서 해당 데이터를 다시 전송하면 이미 새로운 위치 데이터가 만들어졌기 때문에 오래된 정보를 다시 보내는 것이 큰 의미가 없을 수 있다. 이러한 경우에는 이전 데이터가 일부 손실되더라도 최신 데이터를 빠르게 전달하는 것이 중요하다. 음성 통화나 영상 통신도 비슷한 특징을 가

지고 있다. 이전 음성 데이터 하나가 손실되었다고 해서 몇 초 뒤에 해당 음성을 다시 보내는 것은 실시간 통신에서 큰 의미가 없을 수 있다. 따라서 빠른 전송이 중요하며 일부 데이터의 손실을 허용할 수 있는 프로그램에서는 UDP를 사용할 수 있다. 하지만 모든 프로그램에서 UDP가 더 좋은 것은 아니다. 정확성이 중요한 파일 전송이나 중요한 문자열 데이터를 반드시 전달해야 하는 프로그램에서는 TCP가 더 적합할 수 있다. 따라서 UDP와 TCP 중 어떤 방식을 사용할지는 프로그램의 목적에 따라 결정해야 한다.

5. UDP의 장점과 단점

UDP의 가장 큰 장점은 구조가 단순하다는 것이다. TCP처럼 데이터를 보내기 전에 연결 과정을 거치지 않기 때문에 상대방의 IP 주소와 Port 번호만 알고 있다면 바로 데이터를 전송할 수 있다. 이 때문에 프로그램을 구현할 때도 비교적 간단하게 사용할 수 있다. 또 다른 장점은 빠른 데이터 전송이 가능하다는 것이다. UDP는 데이터가 정상적으로 도착했는지 계속 확인하지 않는다. 또한 데이터를 전송한 순서를 관리하거나 손실된 데이터를 자동으로 다시 보내는 과정도 기본적으로 수행하지 않는다. 이러한 과정이 줄어들기 때문에 빠르게 데이터를 전달할 수 있다. UDP는 하나의 프로그램에서 여러 장치로 데이터를 보내는 상황에서도 활용할 수 있다. 네트워크 환경에 따라 브로드캐스트나 멀티캐스트와 같은 방법을 사용하면 여러 장치에 데이터를 전달하는 구조도 만들 수 있다. 또한 데이터를 계속해서 빠르게 보내야 하는 실시간 프로그램에 적용하기 좋다.

하지만 UDP에는 단점도 존재한다. 가장 큰 단점은 데이터의 도착을 보장하지 않는다는 것이다. 송신 프로그램에서 데이터를 정상적으로 보냈다고 하더라도 네트워크 문제로 인해 데이터가 상대방까지 도착하지 않을 수 있다. UDP에서는 이러한 데이터 손실이 발생했을 때 자동으로 다시 보내주지 않는다. 또한 데이터의 순서를 보장하지 않는다. 여러 개의 데이터를 연속해서 보내더라도 먼저 보낸 데이터가 반드시 먼저 도착한다고 보장할 수 없다. 또한 동일한 데이터가 중복으로 도착하는 상황에 대해서도 프로그램에서 필요한 처리를 직접 구현해야 할 수 있다. 이러한 특징 때문에 중요한 데이터를 UDP로 전송해야 하는 경우에는 프로그램에서 별도의 확인 기능을 추가하기도 한다. 예를 들어 데이터를 보낸 후 상대방이 ACK와 같은 확인 메시지를 다시 보내게 만들 수 있다. 송신 측에서 일정 시간 동안 확인 메시지가 들어오지 않으면 데이터를 다시 전송하도록 구현할 수도 있다. 하지만 이러한 기능을 많이 추가하면 UDP를 사용하는 장점인 간단한 구조가 복잡해질 수 있다. 따라서 UDP를 사용할 때는 프로그램의 목적을 먼저 확인해야 한다. 데이터가 일부 손실되더라도 빠르게 최신 데이터를 전달해야 한다면 UDP가 적합하다. 반대로 모든 데이터가 정확하게 전달되어야 한다면 TCP와 같은 다른 통신 방식을 사용하는 것이 더 적절할 수 있다.

7. UDP의 활용 분야

UDP는 빠르게 데이터를 전송해야 하는 다양한 분야에서 사용된다. 대표적인 예로 실시간 영상 전송이 있다. 영상은 짧은 시간 동안 많은 데이터를 계속해서 보내야 한다. 영상 데이터 중 일부가 손실되더라도 다음 영상이 계속 들어오기 때문에 모든 데이터를 다시 보내는

것보다 빠르게 다음 데이터를 보내는 것이 더 중요한 상황이 있다. 음성 통신에서도 UDP를 활용할 수 있다. 실시간 음성 대화에서는 몇 초 전의 손실된 음성 데이터를 나중에 다시 받는 것보다 현재 음성을 빠르게 전달하는 것이 더 중요하다. 온라인 게임에서도 UDP를 사용할 수 있다. 캐릭터의 위치나 방향과 같은 데이터는 계속해서 변경된다. 이전 위치 데이터가 일부 손실되어도 바로 다음 위치 데이터가 도착하면 최신 위치를 사용할 수 있다.

로봇 제어에서도 UDP를 사용할 수 있다. 로봇의 현재 상태나 센서 값 또는 간단한 명령을 빠르게 전송해야 하는 경우 UDP의 빠른 전송 방식이 유용할 수 있다. 예를 들어 로봇의 현재 위치나 카메라 상태를 계속해서 전송하거나 조작 명령을 빠르게 보내는 상황에서 사용할 수 있다. 본 프로그램에서는 문자열 데이터를 주고받는 간단한 채팅 기능을 통해 UDP의 기본적인 송신과 수신 방법을 구현하였다. 하지만 같은 원리를 사용하면 문자열뿐만 아니라 센서 데이터나 로봇 명령과 같은 다른 데이터도 전송할 수 있다. 따라서 UDP 통신의 기본 구조를 이해하면 다양한 네트워크 프로그램을 구현하는 데 활용할 수 있다.

6. UDP 통신을 공부하면서 배운 점

UDP 통신을 공부하면서 가장 먼저 네트워크에서 두 컴퓨터가 데이터를 주고받기 위해서는 단순히 데이터를 보내는 코드만 작성하면 되는 것이 아니라는 것을 알게 되었다. 데이터를 보낼 상대방의 IP 주소와 Port 번호가 필요하며 데이터를 받는 쪽에서도 해당 Port를 열어 데이터를 받을 준비를 해야 한다는 것을 공부하였다.

처음에는 IP 주소와 Port 번호의 차이를 정확하게 알지 못했지만 UDP 통신을 직접 구현하면서 각각의 역할을 이해할 수 있었다. IP 주소는 네트워크에서 데이터를 보낼 컴퓨터를 찾기 위해 사용하고 Port 번호는 해당 컴퓨터 안에서 데이터를 받을 프로그램을 구분하기 위해 사용한다는 것을 알게 되었다.

또한 UDP가 TCP와 다르게 상대방과 연결을 먼저 설정하지 않고 데이터를 바로 전송하는 비연결형 통신 방식이라는 것을 공부하였다. UDP는 데이터를 전송한 후 상대방이 정상적으로 데이터를 받았는지 확인하지 않으며 데이터의 순서나 도착 여부도 보장하지 않는다. 이러한 특징 때문에 데이터의 정확성보다 빠른 전송이 중요한 영상 통신이나 음성 통신 그리고 온라인 게임과 같은 곳에서 UDP가 사용될 수 있다는 것도 알게 되었다.

Qt에서 UDP를 직접 구현하면서 소켓에 대해서도 공부할 수 있었다. 소켓은 프로그램이 네트워크를 통해 데이터를 주고받기 위해 사용하는 통신 수단이며 Qt에서는 QUdpSocket을 사용하여 UDP 통신을 구현할 수 있다는 것을 알게 되었다.

UDP 데이터를 받기 위해서는 bind 함수를 사용하여 프로그램이 사용할 Port를 설정해야 한다는 것도 배웠다. 처음에는 bind가 단순히 UDP를 시작하는 함수라고 생각했지만 실제로는 특정 Port로 들어오는 데이터를 프로그램이 받을 수 있도록 설정하는 역할을 한다는 것을 알게 되었다.

또한 Qt의 Signal과 Slot 기능이 네트워크 통신에서도 사용된다는 것을 공부하였다. QUdpSocket의 readyRead Signal과 데이터를 읽는 함수를 연결하면 UDP 데이터가 들어왔을 때 자동으로 해당 함수가 실행된다. 이를 통해 프로그램에서 계속 데이터를 확인하지 않아도 새로운 데이터가 들어오는 순간 처리할 수 있다는 것을 알게 되었다.

데이터를 보내고 받을 때 사용하는 자료형에 대해서도 공부하였다. Qt의 입력창에서 가져온 문자열은 QString 형태이지만 UDP로 데이터를 전송할 때는 QByteArray 형태를 사용하였다. 따라서 toUtf8 함수를 사용하여 QString을 QByteArray로 변환한 후 전송하였다. 반대로 UDP로 받은 QByteArray 데이터는 QString::fromUtf8을 사용하여 다시 QString으로 변환하였다. 이를 통해 네트워크에서는 문자열을 그대로 전송하는 것이 아니라 바이트 형태의 데이터로 변환하여 전송한다는 것을 알게 되었다.

LocalHost에 대해서도 공부할 수 있었다. 처음에는 LocalHost를 사용하여 데이터를 전송했기 때문에 내가 보낸 데이터가 다시 내 프로그램으로 들어오는 문제가 있었다. LocalHost는 다른 컴퓨터의 주소가 아니라 현재 자신의 컴퓨터를 의미하는 127.0.0.1이라는 것을 알게 되었다. 실제로 두 컴퓨터가 통신하기 위해서는 LocalHost가 아니라 상대방 컴퓨터의 실제 IP 주소를 사용해야 한다는 것을 알게 되었다.

직접 두 컴퓨터 사이에서 메시지를 주고받는 프로그램을 구현하면서 송신과 수신 차이도 이해할 수 있었다. 송신할 때는 전송할 데이터와 상대방의 IP 주소 그리고 Port 번호가 필요하다. 수신할 때는 자신의 Port를 열어 놓고 데이터가 들어오기를 기다린 후 받은 데이터를 읽어야 한다.

이번 UDP 통신 공부를 통해 네트워크 프로그램이 데이터를 보내고 받는 기본적인 과정을 이해할 수 있었다. 이전에는 네트워크 통신이라고 하면 단순히 한 컴퓨터에서 다른 컴퓨터로 데이터를 보내는 것이라고 생각했지만 실제로는 IP 주소와 Port 그리고 소켓과 데이터 변환 과정 등이 필요하다는 것을 알게 되었다

=====

1. 과제1 구현 보고서

과제1 설명: 과제1은 3일차에서 구현한 천지인 키보드를 바탕으로 UDP 통신을 사용해 카카오톡을 구현하는 것이다. 필수 기능은 내가 문자를 입력하면 상대방에게 UDP 통신을 사용해 내가 입력한 문자가 보내지고 또한 상대가 나에게 어떠한 데이터를 보내면 난 그것을 읽어와 채팅창에 출력하여 우리가 현실에서 사용하는 카카오톡과 유사하게 만드는 것이 목표이다.

<구현 기능>

1. 천지인 키보드를 사용해 데이터 입력 받기
2. UDP 통신을 통해 데이터를 보낼 송신부
3. enter 버튼을 클릭 시 입력한 데이터를 상대방에게 전송
4. 채팅창에 전송한 데이터 출력
5. UDP 통신을 통해 데이터를 받을 수신부
5. 채팅창에 받은 데이터 출력

구현 방법

먼저 Qt 내부의 UDP 등을 이용해 네트워크 애플리케이션을 쉽게 만들 수 있도록 해주는 네트워크 모듈을 활성화 해주어야 한다. 그래야 UDP 통신을 할 때 사용되는 헤더파일을 사용할 수 있다. 이 네트워크 모듈을 활성화 시키기 위해서는 Cmake에서 활성화를 시켜주어야 한다.

```
find_package( QT NAMES Qt6 Qt5 REQUIRED COMPONENTS Widgets Network)
find_package( Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets Network)
target_link_libraries( hw1 PRIVATE Qt${QT_VERSION_MAJOR}::Widgets Qt${QT_VERSION_MAJOR}::Network
```

다음과 같이 Cmake에 이 부분들을 찾아 맨 뒤에 Newwork 부분을 추가해준다. 그러면 코드에서는

```
#include <QUdpSocket>
#include <QNetworkDatagram>
#include <QHostAddress>
```

이와 같은 헤더파일을 사용할 수 있게 된다. 다음은 통신에 사용할 IP와 Port 설정해주었다. 본 프로그램에서는 #define을 사용해 사용자의 IP와 상대방의 IP 그리고 PORT 번호도 프로그램의 맨 위에 언제 어디서나 수정이 편리하도록 구성하였다.

```
#define OPERATOR_IP "172.100.5.97"
// 내 컴퓨터의 IP주소
#define ROBOT_IP "172.100.2.99"
// 상대 컴퓨터의 IP 주소
#define TEXT_PORT 10004
// 포트 번호
```

mainwindow에서 UDP 소켓을 생성한다.

```
text_socket = new QUdpSocket(this);
```

소켓을 생성하는 코드는 다음과 같다. 코드에서는 이 소켓 객체를 사용해서 데이터를 주고 받는다. 그리고 UDP로 데이터를 받기 위해서 컴퓨터의 포트를 열고 기다려야 한다. 본 프

그럼에서는 bind() 함수를 사용해서 조건문을 사용해 이 동작을 구현했다.

```
text_socket->bind(QHostAddress::LocalHost, TEXT_PORT)
```

그리고 받은 데이터를 읽는 함수와 데이터를 상태 컴퓨터에 보내는 함수가 있어야 한다. UDP 통신을 사용해서 데이터를 보내는 함수는 다음과 같다.

```
void MainWindow::udp_write( const QByteArray &text, 상대방 포트 번호, QUdpSocket &socket
){
    socket.writeDatagram( 보낼 데이터, QHostAddress(ROBOT_IP), port );
}
```

이 전송함수는 세 가지의 값을 받는다. 전송할 데이터와 전송할 상대방의 포트 번호 전송에 사용할 UDP 소켓을 받고 실제로 상대방에게 데이터를 보낼 때는 UDP 패킷을 보내 데이터를 전송한다. socket.writeDatagram(보낼 데이터, 상대방 IP, 상대방 Port); 소켓은 이런식으로 구현해야 한다.

UDP로 데이터를 받기 위해서는 프로그램이 사용할 포트를 열고 상대방이 데이터를 보내는 것을 기다려야 한다. 이 동작은 bind() 함수를 사용하여 구현하였다.

```
text_socket->bind( QHostAddress::AnyIPv4, TEXT_PORT);
```

bind는 현재 프로그램이 특정 포트를 사용하여 UDP 데이터를 받을 수 있도록 만들어주는 함수이다. TEXT_PORT가 10004(포트 번호)이므로 이 프로그램은 10004번 포트로 들어오는 UDP 데이터를 기다리게 된다. 본 프로그램에서는 bind가 정상적으로 실행되었는지도 확인할 수 있도록 조건문을 사용하였다.

```
if(text_socket->bind(QHostAddress::), TEXT_PORT))
{
    qDebug() << "UDP bind 성공";
}
else
{
    qDebug() << "UDP bind 실패.";
}
```

bind에 성공하면 UDP 데이터를 받을 준비가 정상적으로 된 것이므로 UDP bind 성공이라는 문장을 출력한다. 반대로 포트를 사용할 수 없거나 문제가 발생하면 bind가 실패하게 되고 bind 실패를 확인할 수 있도록 하였다.

UDP는 언제 데이터가 들어올지 모르기 때문에 계속해서 직접 데이터를 확인하는 방법보다 데이터가 들어오면 자동으로 함수를 실행하도록 만드는 것이 편하다. Qt에서는 Signal과 Slot을 사용하여 이러한 동작을 구현할 수 있다.

```
connect( text_socket, &QUdpSocket::readyRead, this, &MainWindow::udp_read);
```

QUdpSocket에는 readyRead라는 Signal이 있다. 상대 컴퓨터에서 UDP 데이터가 들어오면 readyRead Signal이 발생한다. 이 Signal을 udp_read 함수와 연결하였다. 이 방법을 사용하면 데이터가 들어올 때마다 자동으로 udp_read 함수가 실행되기 때문에 프로그램에서 계속 데이터를 확인할 필요가 없다. 또한 상대방이 보낸 UDP 데이터를 읽기 위해 udp_read 함수를 구현하였다.

```
void MainWindow::udp_read()
{
    while(text_socket->hasPendingDatagrams())
    {
        QNetworkDatagram datagram = text_socket->receiveDatagram();
        QByteArray data = datagram.data();
        QHostAddress senderAddress = datagram.senderAddress();
        quint16 senderPort = datagram.senderPort();
        QString text = QString::fromUtf8(data);
        ui->textEdit->setAlignment(Qt::AlignLeft);
        ui->textEdit->append("상대 : " + text);
        qDebug() << "받은 데이터:" << data;
        qDebug() << "보낸 IP:" << senderAddress;
        qDebug() << "보낸 Port:" << senderPort;
    }
}
```

text_socket->hasPendingDatagrams() 이 함수는 현재 아직 읽지 않은 UDP 데이터가 있는지 확인한다. 데이터가 존재하면 true가 되기 때문에 while문 안으로 들어간다. while문을 사용한 이유는 UDP 데이터가 한 번에 여러 개 들어왔을 경우 남아있는 데이터를 모두 읽기 위해서이다.

실제 UDP 데이터는 receiveDatagram 함수를 사용하여 읽는다.

```
QNetworkDatagram datagram = text_socket->receiveDatagram();
```

receiveDatagram은 현재 들어와 있는 UDP 데이터 하나를 읽어서 QNetworkDatagram 형태로 저장한다. QNetworkDatagram에는 실제 데이터뿐만 아니라 데이터를 보낸 사람의 IP 주소와 포트 번호도 같이 들어 있다. 실제 메시지 데이터는 다음 코드로 가져온다.

```
QByteArray data = datagram.data();
```

UDP에서는 문자열을 그대로 보내는 것이 아니라 바이트 형태의 데이터로 전송하기 때문에 받은 데이터의 자료형은 QByteArray가 된다. UDP 데이터를 보낸 컴퓨터의 IP 주소는 다음과 같이 확인할 수 있다.

```
QHostAddress senderAddress = datagram.senderAddress();
```

그리고 데이터를 보낸 쪽의 포트 번호는 다음과 같이 확인한다.

```
quint16 senderPort = datagram.senderPort();
```

이렇게 하면 어떤 IP에서 데이터를 보냈는지 확인할 수 있다.
프로그램을 테스트할 때 UDP 통신이 정상적으로 이루어지고 있는지 확인하기 위해 qDebug를 사용하여 받은 데이터, IP, Port를 출력하도록 구현하였다.

```
qDebug() << "받은 데이터:" << data;  
qDebug() << "보낸 IP:" << senderAddress;  
qDebug() << "보낸 Port:" << senderPort;
```

예를 들어 상대방이 Hello라는 데이터를 보냈다면 실행 화면의 출력창에서 받은 데이터와 상대방의 IP 주소를 확인할 수 있다. 받은 데이터를 문자열로 변환 UDP를 통해 받은 데이터는 QByteArray 형태이기 때문에 채팅창에 문자열로 출력하기 위해 QString으로 변환하였다.

```
QString text = QString::fromUtf8(data);
```

QString::fromUtf8() 함수는 UTF-8 형태의 QByteArray 데이터를 QString으로 변환해주는 함수이다. 이 기능을 사용하면 영어뿐만 아니라 한글도 정상적으로 출력할 수 있다. 즉 데이터가 들어오면 다음과 같은 과정을 거친다.

1. UDP 데이터
2. QByteArray
3. QString::fromUtf8()
4. QString
5. textEdit에 출력

상대방이 보낸 데이터는 채팅 프로그램처럼 왼쪽에 표시되도록 구현하였다.

```
ui->textEdit->setAlignment(Qt::AlignLeft);  
ui->textEdit->append("상대 : " + text);
```

setAlignment를 사용하여 왼쪽 정렬을 설정하고 append 함수를 사용하여 기존 채팅 내용 아래쪽에 새로운 메시지를 추가한다. append를 사용한 이유는 이전에 받은 메시지를 지우지 않고 계속해서 메시지를 추가하기 위해서이다. 예를 들어 상대방이 순서대로 다음과 같이 보냈다고 하면

안녕하세요
테스트

채팅창에는 다음과 같이 출력된다.

상대 : 안녕하세요

상대 : 테스트

UDP 데이터 전송 함수 UDP 통신을 사용해서 데이터를 보내는 함수도 따로 구현하였다.

```
void MainWindow::udp_write( const QByteArray &text, quint16 port, QUdpSocket &socket){
    socket.writeDatagram(text, QHostAddress(ROBOT_IP), port );
}
```

이 전송 함수는 세 가지 값을 받는다. 첫 번째는 실제로 전송할 데이터이다. 두 번째는 데이터를 받을 상대방의 포트 번호이다. 세 번째는 데이터를 전송할 때 사용할 UDP 소켓이다. 그리고 실제 데이터 전송은 writeDatagram 함수를 사용한다.

```
socket.writeDatagram( 보낼 데이터, 상대방 IP, 상대방 Port);
```

UDP에서는 TCP처럼 상대방과 먼저 연결하지 않고 데이터를 보낼 상대방의 IP와 Port를 지정해서 바로 데이터를 전송할 수 있다. 본 프로그램에서는 상대방의 IP가 ROBOT_IP에 저장되어 있기 때문에 다음과 같이 사용하였다.

사용자가 입력한 데이터는 lineEdit에서 가져온다. ui->lineEdit->text() 하지만 lineEdit의 text 함수가 반환하는 자료형은 QString이다. UDP 통신에서는 QByteArray 형태로 데이터를 전송하기 때문에 QString을 QByteArray로 변환해야 한다. 이를 위해 toUtf8 함수를 사용하였다. UTF-8 형태로 변환하기 때문에 영어, 숫자뿐만 아니라 한글도 UDP를 통해 전송할 수 있다. Enter 버튼을 눌렀을 때 데이터 전송 사용자가 lineEdit에 문자열을 입력하고 Enter 버튼을 누르면 입력된 문자열을 가져와 UDP로 전송하도록 구현할 수 있다.

```
void MainWindow::on_btnEnter_clicked()
{
    QByteArray data = ui->lineEdit->text().toUtf8();

    if(data.isEmpty()) return;

    ui->textEdit->setAlignment(Qt::AlignRight);
    ui->textEdit->append(
        QString::fromUtf8(data) + " : 나" );
    text_socket->writeDatagram( data, HostAddress(ROBOT_IP), TEXT_PORT );
    ui->lineEdit->clear();

    qDebug() << "UDP 전송:" << data;
}
```

먼저 lineEdit에 입력한 문자열을 가져온 후 toUtf8을 사용하여 QByteArray로 변환한다. 만약 아무 문자도 입력하지 않았다면 빈 데이터를 보내지 않도록 한다. 그리고 내가 보낸 메시지는 상대방에게 UDP로 전송하는 것과 별개로 내 프로그램의 채팅창에도 직접 출력하도록 구현하였다.

```
ui->textEdit->setAlignment(Qt::AlignRight);
```

```
ui->textEdit->append( QString::fromUtf8(data) + " : 나" );
```

내가 보낸 메시지는 오른쪽 정렬하고 상대방이 보낸 메시지는 왼쪽 정렬하였다.
따라서 채팅창에서는 다음과 같이 구분할 수 있다.

상대 : 안녕하세요

안녕하세요 : 나

상대 : UDP 테스트입니다

잘 됩니다 : 나

그 후 실제 UDP 데이터는 다음 코드를 사용하여 상대 컴퓨터로 전송한다.

```
text_socket->writeDatagram( data, QHostAddress(ROBOT_IP), TEXT_PORT);
```

데이터를 보내고 나면 다음 메시지를 입력할 수 있도록 lineEdit을 비운다.

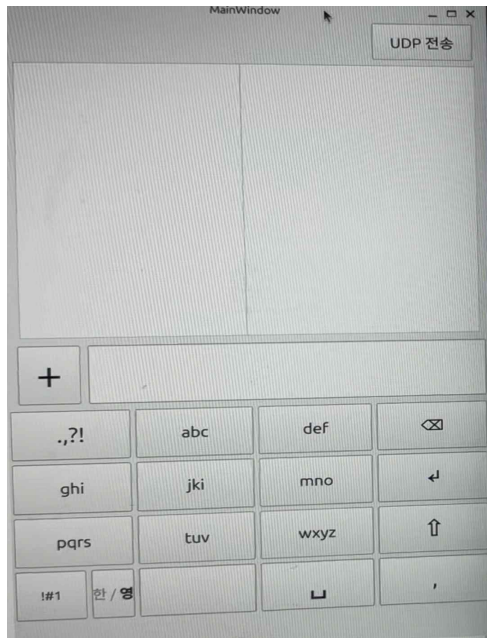
```
ui->lineEdit->clear();
```

카카오톡 프로그램의 대략적인 로직은 다음과 같다.

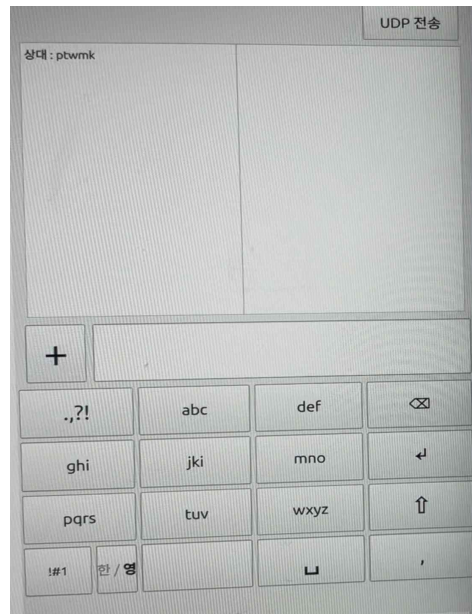
1. 프로그램 실행
2. QUdpSocket 생성
3. bind로 Port 열기
4. readyRead와 udp_read 연결
5. UDP 데이터 수신 대기
6. 사용자가 문자열 입력
7. QString
8. toUtf8()
9. QByteArray
10. writeDatagram()
11. 상대방 IP + Port로 UDP 전송
12. 내 채팅창 오른쪽에 출력
13. 상대방 UDP 데이터 도착
14. eadyRead 발생
15. udp_read 실행
16. receiveDatagram()
17. QByteArray
18. QString::fromUtf8()
19. 내 채팅창 왼쪽에 출력

실행 결과

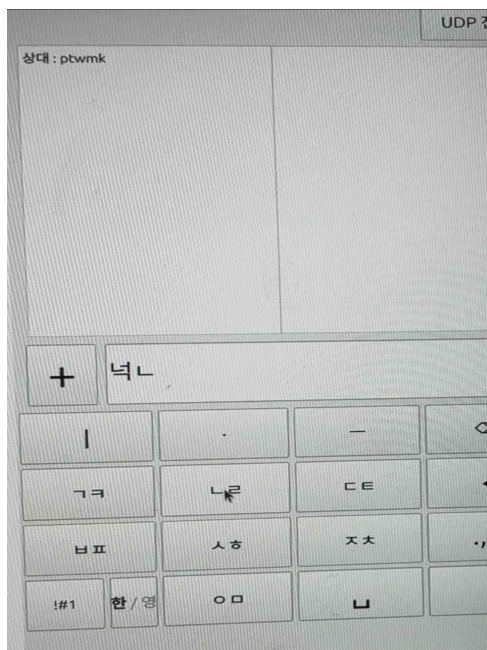
1



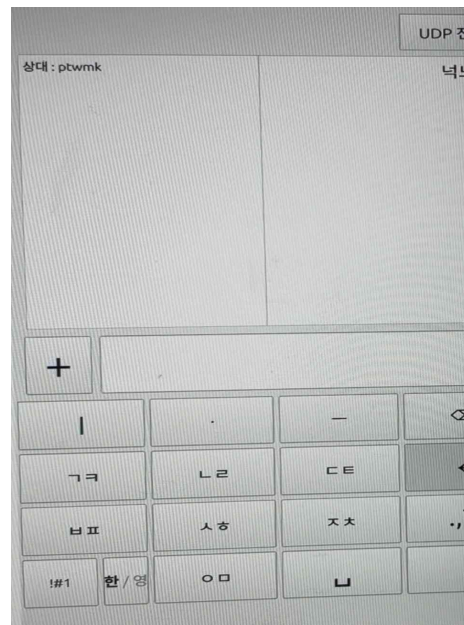
2



3

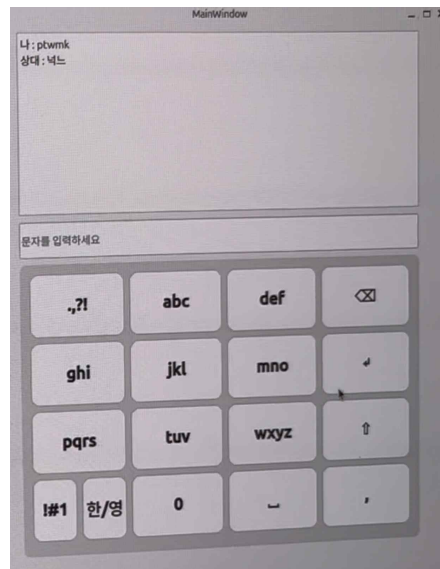


4



먼저 연결할 상대방의 IP를 입력한 프로그램을 실행시키면 1번과 같은 화면이 뜬다. 이 화면이 카카오톡 채팅 화면이고 천지인 키보드가 있는 것을 볼 수 있다. 위에 사각형

이 채팅 공간이다. 상대방이 데이터를 전송하면 채팅 공간에 상대: 메시지와 함께 상대가 보낸 데이터가 함께 출력된다. 그리고 천지인 키보드를 사용해 데이터를 입력한다. 그리고 엔터키를 누르면 데이터가 상대방에게 전송되고 나의 채팅 공간에 내가 입력한 데이터가 출력되는 것을 볼 수 있다. 이러한 시스템을 통해 상대방과 UDP 통신을 통해 대화를 할 수 있다,



위의 사진은 상대방의 ui이다 나의 프로그램에서 전송한 데이터가 상대방에게 잘 전송되었을 볼 수 있다.